

Problem AB23. TRIANGLE

 10 sec.  1024 MB

Die neue Leitung des Nationalkomitees, vertreten durch die Koordinatoren der Gruppen A und B, möchte bei der Auswahl der Nationalmannschaft einen strengen Rahmen für die gestellten Aufgaben beibehalten. Zu diesem Zweck muss eine Geometrieaufgabe gestellt werden. Zudem liegt die Anzahl der bisher gestellten interaktiven Aufgaben unter dem Standard. Um die Krise bei der Auswahl für die IOI zu beheben, hat Sashka beschlossen, eine Aufgabe zu stellen, die sowohl interaktiv als auch geometrisch ist. Sie lautet wie folgt:

Die Jury hat eine Permutation $P_0, P_1, \dots, P_{N-1}$ von $\{1, 2, 3, \dots, N\}$ versteckt. Du musst die Permutation finden. Zu diesem Zweck darfst du der Jury folgende Frage stellen: „Ist es möglich, ein Dreieck mit positiver Fläche mit den Seiten P_A, P_B, P_C zu bilden?“

Beachte, dass (aufgrund der Dreiecksungleichung) bekannt ist, dass dies genau dann möglich ist, wenn:

$$P_A + P_B > P_C, P_A + P_C > P_B \quad \text{und} \quad P_B + P_C > P_A.$$

Schreiben Sie ein Programm **triangle**, das eine Funktion **solve** enthält, die zusammen mit dem Jury-Programm kompiliert wird und mit diesem kommuniziert, indem sie Fragen des oben beschriebenen Typs stellt. Am Ende seiner Ausführung muss es die Permutation ermitteln.

Implementierungsdetails

Sie sollten die Funktion **solve** implementieren:

```
std::vector<int> solve(int N)
```

- N : Länge der Permutation.

Diese Funktion wird pro Test T Mal aufgerufen – einmal pro Subtest, jeweils mit gleichem N – und sollte die verborgene Permutation für den Subtest zurückgeben. Um dies zu erreichen, kann Ihr Programm die Jury-Funktion **query** aufrufen:

```
bool query(int A, int B, int C)
```

- A, B, C : Indizes der Seiten P_A, P_B, P_C .

Die Funktion gibt **true** zurück, wenn ein Dreieck mit positiver Fläche und den Seiten P_A, P_B, P_C existiert, und andernfalls **false**.

Einschränkungen

- $N = T = 1\,000$

Teilaufgabe

**XVII INTERNATIONAL ADVANCED TOURNAMENT IN INFORMATICS
ONLINE 2026**

Teilaufgabe	Punkte	Beschreibung
1	100	Die Teilaufgabe besteht aus 1 Test mit $T = 1\,000$ zufällig generierten Permutationen, die jeweils $N = 1\,000$ Elemente umfassen.

Die Punkte für eine bestimmte Teilaufgabe werden nur vergeben, wenn alle Tests dafür erfolgreich bestanden wurden.

Punktevergabe

Sei $Q_{contestant}$ die durchschnittliche Anzahl von Abfragen, die Ihr Programm bei einem Aufruf von `solve` für den einzelnen Test durchführt, und sei $Q_{target} = 8770$. Dann beträgt Ihre Punktzahl für die Aufgabe:

$$\begin{cases} 0 & \text{wenn } Q_{contestant} > 2 \times 10^6, \\ 100 & \text{wenn } Q_{contestant} \leq Q_{target}, \\ 100 \times \max \left(0, 15, 1 - \sqrt{1 - \left(\frac{Q_{target}}{Q_{contestant}} \right)^{0,55}} \right) & \text{wenn } Q_{contestant} > Q_{target}. \end{cases}$$

Beispielinteraktion

Für diese Beispielinteraktion gilt $N = 4$, $T = 2$.

Aktion des Teilnehmers	Aktion der Jury
	<code>solve(4)</code>
<code>Dreieck(0, 0, 0)</code>	wahr
<code>Dreieck(0, 1, 2)</code>	falsch
<code>Dreieck(0, 1, 3)</code>	falsch
<code>Dreieck(0, 2, 3)</code>	wahr
<code>Dreieck(1, 2, 3)</code>	falsch
<code>return {3, 1, 2, 4};</code>	
	<code>solve(4)</code>
<code>triangle(0, 1, 2)</code>	false
<code>triangle(0, 1, 3)</code>	true
<code>triangle(0, 2, 3)</code>	false
<code>triangle(1, 2, 3)</code>	false
<code>return {4, 2, 1, 3};</code>	

Lokale Tests

Eingabeformat:

- Zeile 1: drei ganze Zahlen T , N und R – die Anzahl der Teilprüfungen, die Größe der Permutationen und der Ausführungsmodus. Wenn $R = 1$ ist, generiert der lokale Bewerter gleichverteilte Zufallspermutationen und erwartet in der zweiten Zeile eine Zahl – S –, die als Startwert für seinen Zufallsgenerator dient. Wenn $R = 2$ ist, setzt sich die Eingabe wie folgt fort:
- Zeilen 2 bis $(1 + T)$: Permutationen von $\{1, 2, \dots, N\}$.

Ausgabeformat:

- Zeile 1: eine Fehlermeldung oder die durchschnittliche Anzahl der Abfragen für die Teilprüfungen, falls alle Permutationen korrekt identifiziert wurden.